



華中科技大學

第二章 数据的机器级表示和处理



主要内容

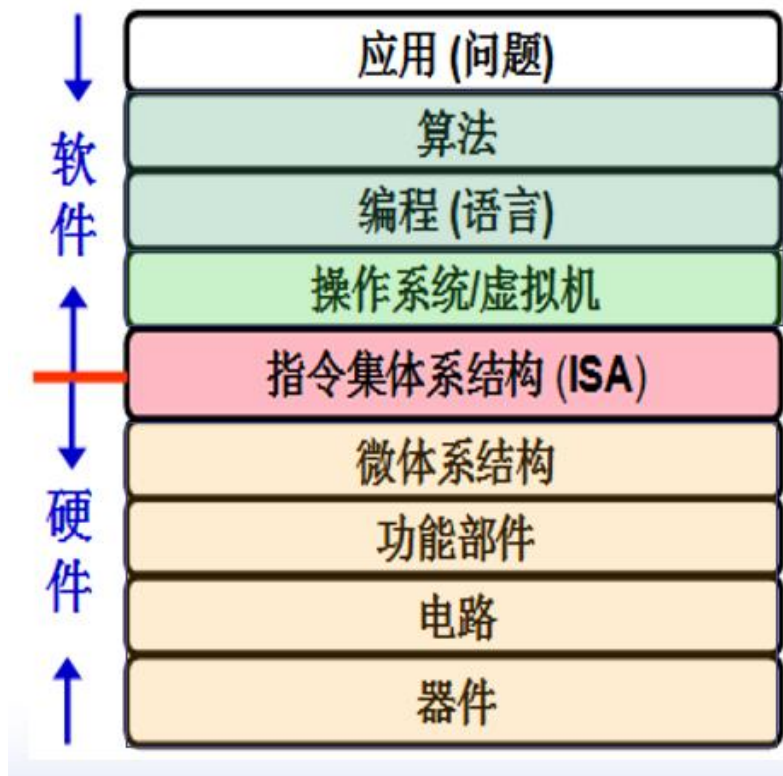


华中科技大学

- ◆ 2.1 编码和数制
- ◆ 2.2 数值数据的机内表示
- ◆ 2.3 字符数据的机内表示



一、计算机系统层次中的编码任务



- 在不同的层次中实现各自的编码
- 在不同的层次间实现编码的映射Integrator And Computer



一、计算机系统层次中的编码任务

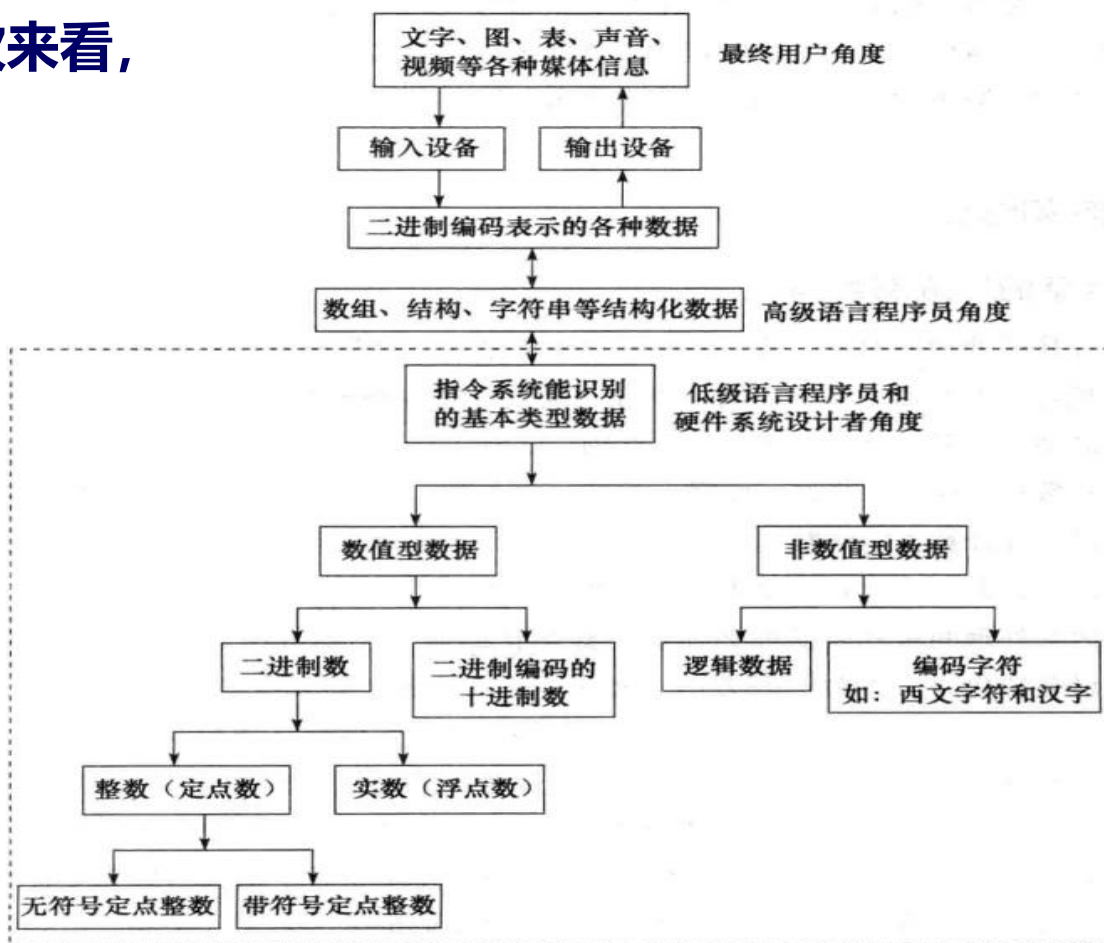
例如：

- C语言（高级编程语言）中，整型数据按长度分为short、int、long；按有无符号分为有符号整数int和无符号整数unsigned int。
- 汇编语言（ISA层）中，整型数据按长度分为BYTE、WORD、DWORD等类型。汇编语言中的数据本身只考虑编码的长短，不考虑有无符号。这也是ISA以下的硬件层的整型数据的编码方案。（暂称ISA层中的一个编码为一个码点）。
- 可以采用补码、原码等编码方案，将高级编程语言中的有符号整数，映射为ISA层中指定长度的码点。



二、外部信息与内部数据的转换

在从不同的角度和层次来看，
数据有不同的形态





二、外部信息与内部数据的转换

- 外部形式： 数值、文字、图、声音、视频等
- 算法： 图、表、树、队列、矩阵等
- 程序员： 数组、结构、指针、实数、整数、布尔数、字符、字符串等
- ISA： 整数、浮点数、位串等。

(汇编语句中，操作数的类型由指令决定，只看操作数的话，则只有“码点”)



三、码点和码点空间

1. 数据存储的单位和宽度

- bit: 数据的每一位 (0或1) , 是二进制信息的最小单位
- byte: 每8个bit组成一个字节。是计算机中的内存的寻址单位
- word: 两个byte组成一个字
- dword: 四个byte组成一个双字





2. 存储容量单位

- KB: 2^{10} 字节
- MB: 2^{20} 字节
- GB: 2^{30} 字节
- TB: 2^{40} 字节
- PB: 2^{50} 字节



3. 码点空间

针对一个位数为 n 的存储单元：

- 将其一个特定的状态称为一个“码点”
- 将其所有的状态的集合，称为其“码点空间”
- 则每个码点可以用一个 n 位二进制数表示
- 为了书写阅读方便，这个二进制数也可以用一个对应的16进制数表示，或对应的10进制数表示。





3. 码点空间

存储单元的状态数目:

- 1 bit的存储, 提供 $2^1 = 2$ 个不同状态
- 1 byte (8 bit)的存储, , 提供 $2^8 = 256$ 个不同状态
- 1 word (16 bit)的存储, , 提供 $2^{16} = 65536$ 个不同状态
- 1 dword (32 bit)的存储, , 提供 $2^{32} = 4294967296$ 个不同状态
- n bit的存储, 提供 2^n 个不同的状态



3. 码点空间



华中科技大学

例如：一个16位的存储单元，其“码点”为：

二进制	十六进制	十进制
0000 0000 0000 0000	0000H	0
0000 0000 0000 0001	0001H	1
.....		
1010 1011 1012 1013	0ABCDH	43981
.....		
1111 1111 1111 1111	0FFFFH	65536





4. 码点的语义和映射

从ISA的层次来理解：

- 存储单元的“码点”的编码，只是为了区分存储单元的不同状态。
- 存储单元的“码点”的编码，不具有语义上的含义。
- 存储单元的“码点”，需要从软件层次出发，可以和上层软件编程中的语义数值进行一一对应。例如一个“码点”，进行对应后，可以表示一个整数、一个正数、一个负数、一个浮点数、或一个字符。
- 对存储单元的语义的指定，是依靠在指令中，使用不同的操作符实现的。

例如：

- ◆ IMUL AX; 有符号乘法，将AX看作有符号数
- ◆ MUL AX; 无符号乘法，将AX看作无符号数



四、编码方案

将语义数据和“码点”进行映射，即对数据进行编码。不同的映射方案，即不同的编码。例如：

- 补码
- 原码
- BCD码
- 浮点数编码
- ASCII码
- GBK码
- Unicode码

主要内容



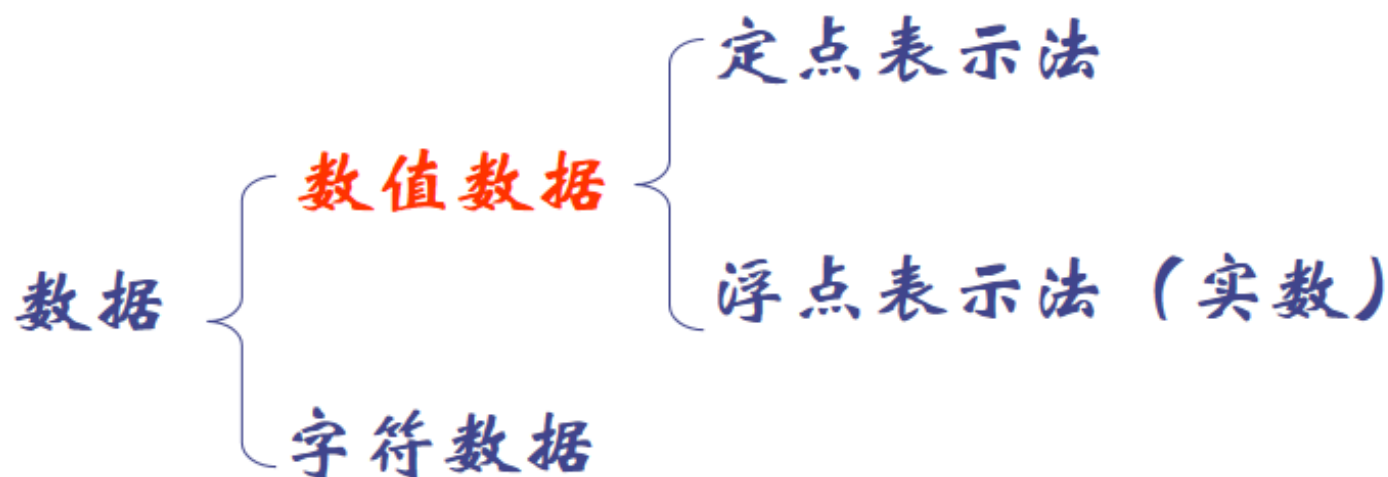
华中科技大学

- ◆ 2.1 编码和数制
- ◆ 2.2 数值数据的机内表示
- ◆ 2.3 字符数据的机内表示





2.2. 数值数据的机内表示





一、有符号整数

- 一个包含正负整数的集合，例如 $\{-128, -127, \dots, 0, \dots, 127\}$ 。
- C语言中short、int、long类型表示的整数。
- 有符号数一般用补码表示



一、有符号整数

1. 正数的补码:

它本身的二进制数，最高位一定为0。

如：45用8位补码表示：0010 1101

2. 负数的补码:

负数X的补码 = $2^n - |X|$ ，其中n为字长。（最高位为1，表示负数）。

如：n = 8时，-45的补码为：1101 0011 ($256 - 45 = 211$)。

简单方法：X的补码 = |X|的二进制数取反 + 1。





3. 补码的表示范围

字长为 n 的补码表示范围:

$$-2^{(n-1)} \sim +2^{(n-1)} - 1$$

$$n = 8 \text{ 时, } -128 \sim 127 \text{ 或 } -2^7 \sim 2^7 - 1$$

$$n = 16 \text{ 时, } -32768 \sim 32767 \text{ 或 } -2^{15} \sim 2^{15} -$$

1

(码点的一半表示负数, 一半表示正数; 0看做正数)





4. 补码的运算特点

- 补码的加法

$$[m + n]_{\text{补}} = [m]_{\text{补}} + [n]_{\text{补}}$$

- 相反数的补码：-n的补码，等于对 n 的补码进行求补运算：

$$[-n]_{\text{补}} = [n]_{\text{补}} \rightarrow \text{求补}$$

- 补码的减法：减去一个数n(的补码)，等于加上n的相反数(的补码)：

$$[m - n]_{\text{补}} = [m]_{\text{补}} + [-n]_{\text{补}}$$





二、无符号整数

- 一个只包含正整数的集合，例如 $\{0, 1, 2, \dots, 255\}$
- C语言中unsigned short, unsigned int, unsigned long类型表示的无符号整数
- 无符号数用原码表示



原码的表示范围

字长为 n 的原码表示范围: $0 \sim 2^n - 1$

例:

$n = 8$ 时, $0 \sim 255$

$n = 16$ 时, $0 \sim 65535$



三、浮点数

二进制小数:

$$\pm (a^n 2^n + \dots + a^1 2^1 + a^0 2^0 + a^{-1} 2^{-1} + \dots + a^{-m} 2^{-m} + \dots)$$

例如:

$$1.25 = 1.01B;$$





规格化数据

$$\pm 1.XXXXXB * 2^n$$

- 科学表示法：小数点左边恒为1
- 要素：
 - ◆ 正负号
 - ◆ 尾数 XXXX
 - ◆ 指数 n
- 指数：以2为底的指数，指数值可以为正或负。



单精度浮点数

31	30	23	22	0
符号	8 位指数			23 位尾数

- 符号位：0表示正数，1表示负数
- 指数：一个字节用来表示指数。

指数是采用的“移码”来表示的： $n - 127$

指数码点	指数值
0000 0000	特殊值，尾数为0时表示0；否则表示一个非规格化的小数（ $2^{-126}(0.f)$ ）
0000 0001	-126
n	$n - 127$
1111 1110	+127
1111 1111	特殊值，尾数为0时表示 ∞ ；否则表示NaN



单精度浮点数



华中科技大学

- 尾数：规格化数据中，尾数值为 1.XXXX

例：

`float d1 = -0.75;`

$(-0.75) = -0.11\text{B} = -1.1\text{B} * 2^{-1}$

符号位：1

指数： $n - 127 = -1$, $n = 126 = 0111\ 1110\text{B}$

尾数：1B

d1的二进制表示：1 0111 1110 1000 0000 0000 0000
0000 000B

对应的4字节内容：BF F4 00 00 00 00 00 00H





四、十进制数

用二进制数来表示十进制数，80X86提供直接处理BCD码的指令。

如： $98 = 1001\ 1000\text{BCD}$

压缩BCD码： $9781 = 1001\ 0111\ 1000\ 0001$

非压缩BCD码： $9781 = 00001001\ 00000111$
 $00001000\ 00000001$

主要内容



华中科技大学

- ◆ 2.1. 编码和数制
- ◆ 2.2. 数值数据的机内表示
- ◆ 2.3. 字符数据的机内表示



2.3. 字符数据的机内表示



华中科技大学

字符编码：字符集到“码点空间”的映射



一、西文字符的编码

西文字符采用ASCII码表示

十进制	→	0	16	32	48	64	80	96	112	128	144	160	176	192	208	224	240
↓	十六进制	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	①	▶	②	0	@	P	'	p	Ç	É	á	⌘	⌘	∞	≡	
1	1	⊙	◀	!	1	A	Q	a	q	ü	æ	í	⌘	⌘	⌘	β	±
2	2	●	‡	"	2	B	R	b	r	é	Æ	ó	⌘	⌘	⌘	Γ	≥
3	3	♥	!!	#	3	C	S	c	s	ˆ	ô	ú		⌘	⌘	Π	≤
4	4	♦	¶	\$	4	D	T	d	t	ā	ö	ñ	⌘	⌘	⌘	Σ	∫
5	5	♣	§	%	5	E	U	e	u	à	ò	Ñ	⌘	⌘	⌘	σ	∫
6	6	♠	—	&	6	F	V	f	v	á	û	ä	⌘	⌘	⌘	μ	÷
7	7	③	‡	'	7	G	W	g	w	ç	ù	ö	⌘	⌘	⌘	τ	≈
8	8	■	↑	(	8	H	X	h	x	ê	ÿ	¿	⌘	⌘	⌘	Φ	°
9	9	○	↓	)	9	I	Y	i	y	ë	Ö	⌘	⌘	⌘	⌘	θ	•
10	A	④	→	*	:	J	Z	j	z	è	Ü	⌘	⌘	⌘	⌘	Ω	•
11	B	♂	←	+	:	K	[	k	{	ì	Ç	½	⌘	⌘	⌘	δ	√
12	C	♀	⌘	,	<	L	\	l		í	£	¼	⌘	⌘	⌘	∞	∞
13	D	⑤	↔	—	=	M	]	m	}	î	¥	ï	⌘	⌘	⌘	∅	²
14	E	♂	▲	•	>	N	^	n	~	Ä	ß	«	⌘	⌘	⌘	∈	■
15	F	☒	▼	/	?	O	_	o	Δ	Å	f	»	⌘	⌘	⌘	∩	⑥



華中科技大學

二、汉字字符的编码

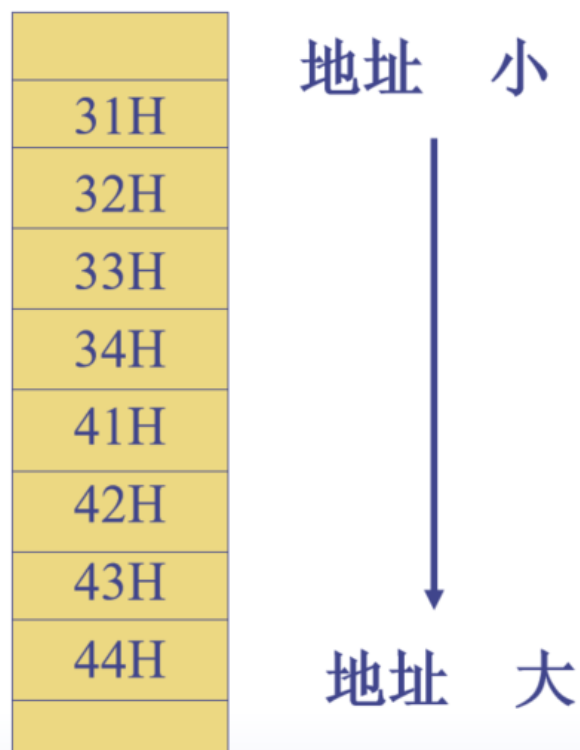
- 汉字字符集
- 汉字字符的编码
 - ◆ GBK
 - ◆ Unicode





三、字符串的存放

字符串“1234ABCD”的存放结构



三、字符串的存放

例：在键盘上输入123。计算机中得到是什么呢？

若要使用其作数值运算，如何处理呢？

